case are optional (and in fact there's an official coding standard that prohibits closing tags in pure PHP files; we'll explore coding standards in a bit). This will probably make more sense if we start writing some PHP, but before we do that, we'll launch another web server. In the interests of balance, we'll use an Apache environment; it's time for more Docker…

PHP Embedded in HTML

No matter which SAPI it's running, the PHP interpreter will emit any text in source code that is not contained in PHP tags directly as output. The idea is (or was) that you can mainly write declarative HTML, dipping into PHP code only when imperative programming is essential. The syntax of PHP tags deliberately resembles HTML tags for this reason, and there will be more said on this programming style in Chapter 11, "Programming for the Internet," where I will do my utmost to dissuade you from ever using it!

Anyway, back to the matter at hand, we're going to launch an old-school web server: Apache with PHP embedded inside (via `libphp.so`), for which there's an official Docker image:

```
$ docker run -p 8080:80 --rm --name apache_mod_php -v "$PWD":/var/www/html
php:8.4-apache
...
[...] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.65 (Debian)
PHP/8.4.11 configured -- resuming normal operations
[...] [core:notice] [pid 1:tid 1] AH00094: Command line: 'apache2 -D
FOREGROUND'
...
```

This gets us an Apache2 server, running some flavor of the `mod_php` SAPI (for this particular PHP 8.4 image, the `PHP_SAPI` string is actually `apache2handler`, but it's the same thing). It is running in the foreground, so our shell is now busy receiving `STDOUT` and `STDERR` from the container (the `php:8.4-apache` image is configured to send Apache and PHP log output to `/dev/stdout` and `/dev/stderr` inside the container, and the container is configured to send those to the host's equivalent devices).